

Numerical calculus quick-start

MCSB Bootcamp — Mathematical and Computational Track

Jun Allard

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
```

The coffee bean model

A jar holds N scoops of coffee. Each day you drink one scoop's worth and top the jar back up with decaf, so the fraction of the jar that is still caffeinated shrinks by a factor of $1 - 1/N$ every day.

```
N = 10 # number of scoops in each jar
n_max = 2 * N # max number of days to simulate

x = np.zeros(n_max) # fraction caffeinated
x[0] = 1.0 # initial fraction caffeinated

for n in range(1, n_max):

    x[n] = (1 - 1 / N) * x[n - 1]

# finished loop through days

days = np.arange(1, n_max + 1)

fig, ax = plt.subplots()
ax.plot(days, x, "-ok")
ax.set_ylabel("Fraction caffeinated")
ax.set_xlabel("Days")
plt.show()
```

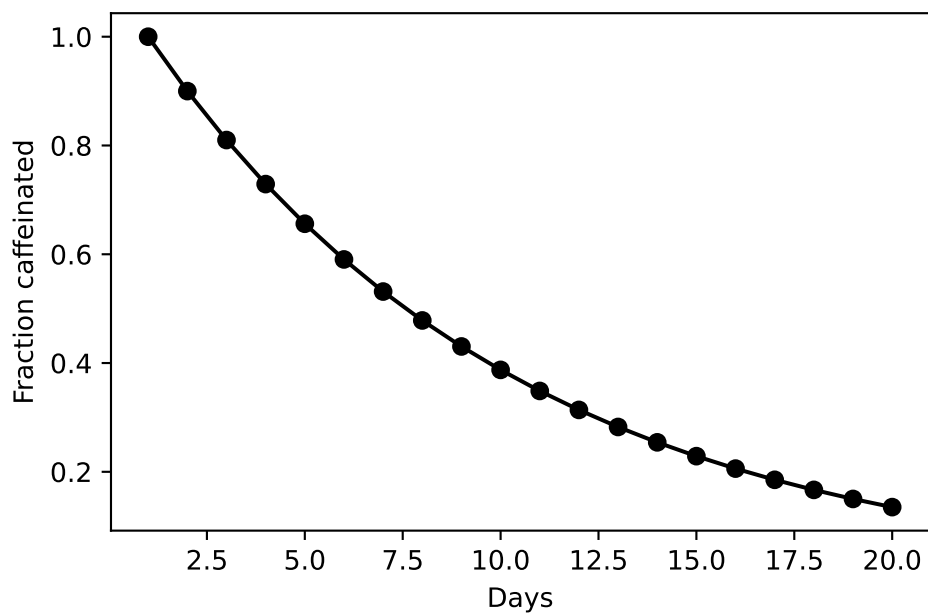


Figure 1: Fraction of the jar still caffeinated, day by day, for a ten-scoop jar.

The differential equation

There are two ways to think about a differential equation. One is as a relationship between the derivatives of $u(t)$ and the function itself. A population of *E. coli* in a large petri dish:

$$\frac{du}{dt} = 5u$$

A charged biomolecule flying through a mass spectrometer:

$$m \frac{d^2u}{dt^2} = -\zeta \frac{du}{dt} + qE_0$$

Any function $u(t)$ either satisfies the differential equation or it does not. Sometimes it is hard to guess the solution.

$$u(t) = 27.3 \exp(5t) \quad u(t) = 1000 \exp(5t) \quad u(t) = 12 \exp(-3t)$$

$$u(t) = (1 - \exp(-\zeta t/m)) \frac{qE_0}{m} t$$

One differential equation often has many solutions.

A differential equation involving a function of one variable, e.g. time t , is called an ordinary differential equation (ODE).

The set of models $du/dt = f(u)$, $u(0) = u_0$

The second way of thinking about a differential equation is as a rule that tells you, given the state of a system at time t , how it is going to change.

In this case, for the model to make a prediction, we need to state an initial condition. The combination of differential equation and initial condition is called an *initial value problem* or *ivp*.

A gene produces protein at a constant rate a (copies per hour). The proteins are removed at a per-molecule rate b (1/hrs).

$$\frac{dP}{dt} = a - bP$$

```
# dP/dt = a - b*P
```

If the protein promotes its own expression, then one model of this is:

$$\frac{dP}{dt} = (a_0 + a_1 P) - bP$$

```
# dP/dt = (a_0 + a_1*P) - b*P
```

If the protein promotes its own degradation, then one model of this is:

$$\frac{dP}{dt} = a - (b_0 + b_1 P) P$$

```
# dP/dt = a - (b_0 + b_1*P)*P
```

Question: why did the modeller choose to put an asymmetry between production (constant) and removal (linear)?

Numerical ODEs

Euler method, and more complicated methods

Thinking about an ODE as a rule that states how a system will change, there is an obvious way to simulate numerical solutions: pick a small dt and step forward just as you would for a discrete-time system. This is the Forward Euler method.

```
dt = 0.01 # time step

a_0 = 500 # molecules per hour
a_1 = 1 # molecules per hour, per existing molecule of A
b = 4 # 1/hrs

nt_max = 200

P = np.zeros(nt_max + 1)
t_array = np.linspace(0, nt_max * dt, nt_max + 1)

P[0] = 0 # initial condition

for nt in range(nt_max):

    dPdt = (a_0 + a_1 * P[nt]) - b * P[nt] # "dPdt" is a convenient shorthand for "dP/dt"

    P[nt + 1] = P[nt] + dPdt * dt

# finished loop through time

# plot it
fig, ax = plt.subplots()
ax.plot(t_array, P)
ax.set_ylabel("Molecules of protein A")
ax.set_xlabel("Time (hours)")
plt.show()
```

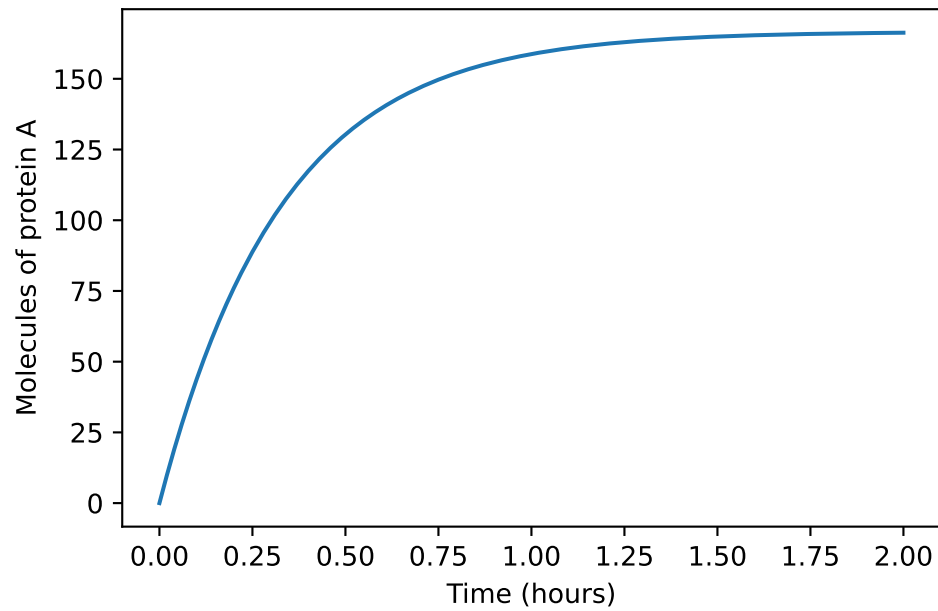


Figure 2: Forward Euler solution of the self-activating gene model, at $dt = 0.01$ h.

Solving an ODE with a library solver

```
def rhs(t, P):
    """The model, in the form solve_ivp asks for: given the time and the protein level P, how
    return (a_0 + a_1 * P) - b * P

sol = solve_ivp(rhs, [0, 2.0], [0.0], t_eval=np.linspace(0, 2.0, 1000))

T = sol.t
P = sol.y[0]

fig, ax = plt.subplots()
ax.plot(T, P)
ax.set_ylabel("Molecules of protein A")
ax.set_xlabel("Time (hours)")
plt.show()
```

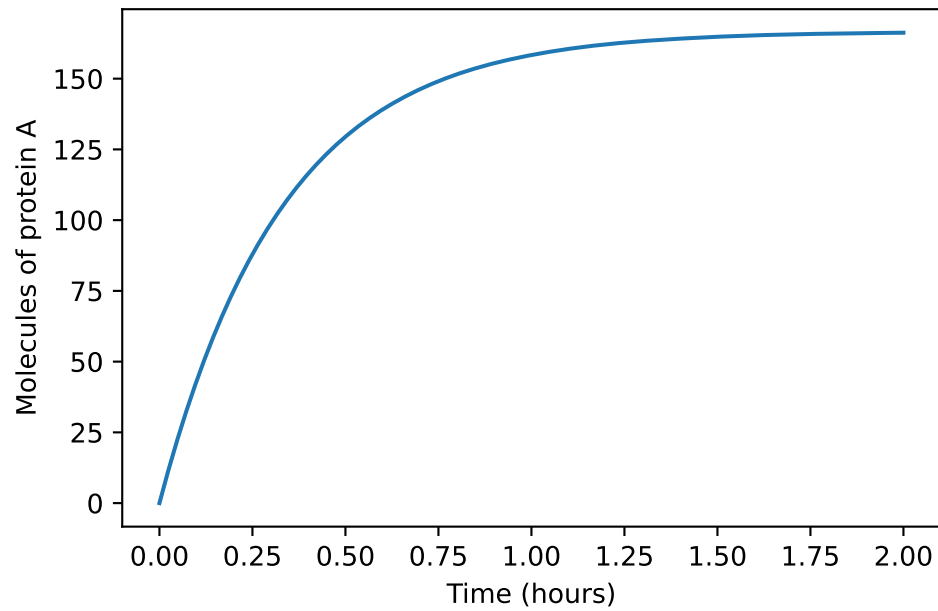


Figure 3: The same model solved with an adaptive Runge–Kutta method.

Let's explore the solution for different values of a_1 .

```
fig, ax = plt.subplots()
for a_1 in [0, 0.1, 0.5, 1, 3, 5, 7, 10]:

    # t_eval asks the solver for the solution on a fine grid rather than only at the steps i
    sol = solve_ivp(rhs, [0, 2.0], [0.0], t_eval=np.linspace(0, 2.0, 1000))

    ax.plot(sol.t, sol.y[0], label=f"$a_1 = \{a_1\}$")

ax.set_ylim(0, 1000)
ax.set_ylabel("Molecules of protein A")
ax.set_xlabel("Time (hours)")
ax.legend(fontsize="small")
plt.show()
```

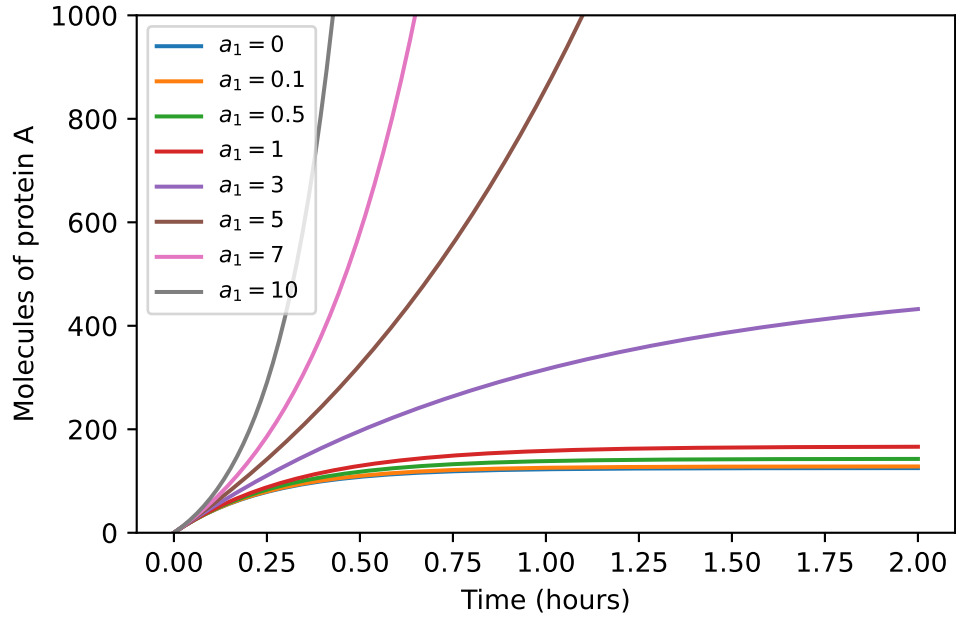


Figure 4: The self-activating gene model for eight strengths of feedback.

Analytical examples

Linear ODEs that look like

$$\frac{du}{dt} = a + bu$$

have solutions

$$u(t) = -\frac{a}{b} + C \exp(bt)$$

where the single constant C is fixed by the initial condition — one first-order equation, one arbitrary constant. If $b > 0$, exponential growth; if $b < 0$, exponential decay.

Now take

$$\frac{dx}{dt} = x^2, \quad x(0) = 0.2$$

whose solution is

$$x(t) = \frac{1}{5-t}.$$

It reaches infinity at $t = 5$, which is called finite time blow-up. Ask a numerical solver to integrate to $t = 6$ and it cannot get there.

```
def quadratic_rhs(t, x):
    return x**2

sol = solve_ivp(quadratic_rhs, [0, 6.0], [0.2])

print(sol.success)
print(sol.message)
print(f"got as far as t = {sol.t[-1]:.4f}, where x = {sol.y[0, -1]:.3e}")
```

False

Required step size is less than spacing between numbers.

got as far as t = 4.9999, where x = 1.023e+14

Multivariate differential equations

If there are multiple quantities to track — multiple molecules, or multiple species — then an ODE model may be a system of coupled ODEs:

$$\frac{du}{dt} = f(u, w), \quad \frac{dw}{dt} = g(u, w), \quad u(0) = u_0, \quad w(0) = w_0$$

Here is gene expression where the protein is an activator: u is RNA, w is protein.

```
def dxdt(t, state):
    """du/dt and dw/dt for the RNA-protein system, given the state [u, w]."""
    u, w = state

    du_dt = -10 * u + 6 * w + 0.1
    dw_dt = -2 * w + 1.9 * u

    return [du_dt, dw_dt]

sol = solve_ivp(dxdt, [0, 24], [0.0, 0.0], t_eval=np.linspace(0, 24, 1000))
```



```

T = sol.t
rna, protein = sol.y

fig, ax = plt.subplots()
ax.plot(T, rna, "-r")
ax.plot(T, protein, "--", color=[0.5, 0, 1])
ax.set_ylabel("Molecular concentration (micromolar)")
ax.set_xlabel("Time (hours)")
plt.show()

```

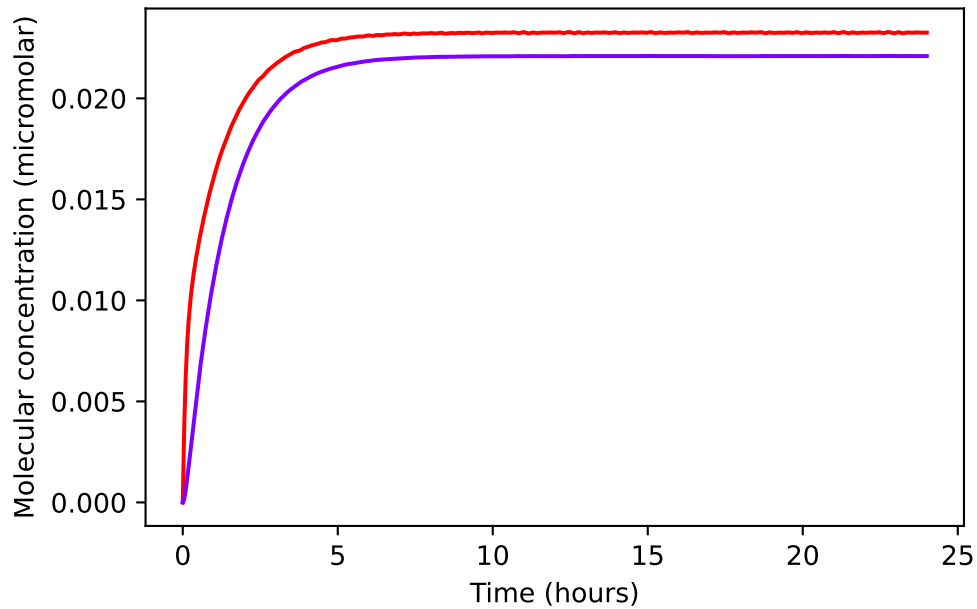


Figure 5: RNA and protein concentrations approaching their steady state.

Notes:

- `solve_ivp` hands your function the whole state as one array and expects the whole set of derivatives back as one list. It then gets unpacked with `u, w = state` on the first line. This lets the two equations below be easier to read.
- `solve_ivp` returns `sol.y` with one *row* per variable, so unpacking it with `rna, protein = sol.y` gives one array per variable, the same way `u, w = state` does inside the function.